

DataShare — Support de présentation

Mission : Pilotez le développement d'une solution informatique

Format : 15 minutes de présentation

Structure : Contexte → Choix tech → Architecture → Démo → Doc → IA

SLIDE 1 — Page de garde

DataShare

Plateforme de transfert sécurisé de fichiers

MVP — Prototype investisseurs

Avril 2026

SLIDE 2 — Le problème métier

Contexte

DataShare veut permettre aux **freelances et petites entreprises** d'envoyer des fichiers de façon simple et sécurisée.

Le besoin en 3 points

1. **Envoyer** un fichier sans compte email ni FTP
2. **Partager** un lien unique avec expiration automatique
3. **Contrôler** l'accès (mot de passe optionnel, historique)

Périmètre MVP (4 semaines)

- ✓ Création de compte et connexion
- ✓ Upload de fichiers (max 50 Mo)
- ✓ Lien de téléchargement unique avec expiration
- ✓ Historique et suppression
- ✓ Protection par mot de passe (optionnel)

SLIDE 3 — Choix technologiques

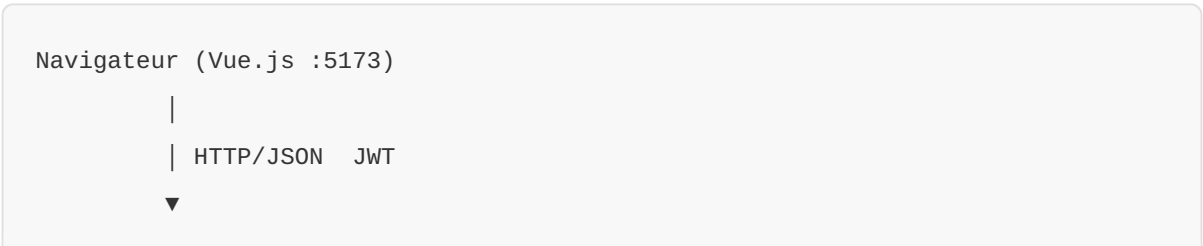
Stack retenu

Composant	Technologie	Pourquoi
Back-end	Django 5.2 + DRF	Batteries incluses, ORM robuste, productivité maximale
Auth	JWT (simplejwt)	Stateless, compatible SPA, standard API
Front-end	Vue.js 3 + Vite	Composition API puissante, écosystème cohérent
State	Pinia	Officiel Vue 3, API simple
BDD	PostgreSQL 16	UUID natif, standard professionnel
Stockage	Disque local	Suffisant pour le prototype
Infra	Docker Compose	Reproductibilité, onboarding en 1 commande

Décision clé : architecture découplée

- Frontend ↔ Backend séparés = API réutilisable (mobile, partenaires)
- Déploiement indépendant de chaque couche
- Facilite la montée en charge

SLIDE 4 — Architecture





Sécurité dans l'architecture

- Token JWT dans chaque requête protégée
- Isolation stricte : un utilisateur ne voit que ses fichiers
- Liens de partage = UUID v4 (128 bits, non devinables)
- Fichiers accessibles uniquement via token opaque, jamais en accès direct

SLIDE 5 — Modèle de données

2 entités principales

User (Django built-in) - `id`, `username`, `email`, `password` (haché PBKDF2)

SharedFile - `id` — UUID primaire - `owner` → User (clé étrangère) - `original_name`, `size`, `content_type` - `share_token` — UUID unique pour le lien de partage - `expires_at` — expiration configurable (24h / 3j / 1 semaine) - `password_hash` — protection optionnelle (PBKDF2) - `created_at`

Règles métier

- Taille max : **50 Mo** (validée serveur + client)
- Expiration : **24h par défaut**, choix utilisateur à l'upload
- Lien expiré → HTTP 410 (Gone), fichier toujours en base

SLIDE 6 — Démonstration

Parcours utilisateur complet

1. **Page d'accueil** → "Tu veux partager un fichier ?"
2. **Inscription / Connexion** → formulaire, JWT stocké

3. **Dashboard** → "Mon espace" avec sidebar
4. Ajouter un fichier (glisser-déposer ou clic)
5. Choisir l'expiration et un mot de passe optionnel
6. Copier le lien de partage généré
7. **Page de téléchargement** (lien public)
8. Badge d'expiration (jours restants)
9. Saisie du mot de passe si protégé
10. Téléchargement natif du fichier

Gestion des erreurs

- Fichier trop grand → message immédiat côté client
- Mot de passe incorrect → 403 avec message clair
- Lien expiré → 410 + message explicatif
- Token expiré → refresh automatique transparent

SLIDE 7 — Documentation technique

Ce qui est livré

Fichier	Contenu
<code>README.md</code>	Installation en 2 commandes, endpoints, structure
<code>docs/openapi.yaml</code>	Spec OpenAPI 3.0 complète de tous les endpoints
<code>TESTING.md</code>	Plan de tests, 38 cas, 100% de couverture
<code>SECURITY.md</code>	Scan pip-audit, décisions documentées
<code>PERF.md</code>	Mesures réelles, script k6, budget front
<code>MAINTENANCE.md</code>	Procédures MAJ, sauvegardes, surveillance

API — Exemple d'usage

```
# Login
curl -X POST localhost:8000/api/auth/login/ \
  -d '{"username":"user","password":"pass"}
```

```
# Upload
curl -X POST localhost:8000/api/files/upload/ \
  -H "Authorization: Bearer <token>" \
  -F "file=@rapport.pdf" -F "expiry_hours=72"
```

SLIDE 8 — Qualité et tests

Résultats

Indicateur	Valeur
Tests unitaires + intégration	38 tests
Couverture de code	100%
Seuil cible	70%
Vulnérabilités applicatives	0

Ce qui est testé

- Inscription, connexion, profil
- Upload (succès, trop grand, sans fichier, avec mot de passe, expiration custom)
- Suppression (propre fichier, isolation utilisateur)
- Téléchargement (valide, expiré, mot de passe correct/incorrect)
- Modèle (is_expired, is_password_protected, **str**)

Exécution

```
cd backend && venv/bin/pytest
# → 38 passed, 100% coverage
```

SLIDE 9 — Utilisation de l'IA

Posture : copilote technique, pas vibe coding

L'IA (Claude — Anthropic) a été utilisée comme **junior assigné à une User Story précise**, avec supervision systématique.

US pilotée par l'IA : Upload + Partage

Tâches confiées : - Modèle `SharedFile` avec hachage de mot de passe - Vues DRF (upload, download, info publique) - Store Pinia et composants Vue.js - Suite de tests pytest

Mon rôle de supervision

Action	Exemple concret
Définir les tâches	Prompt structuré : comportement attendu + cas d'erreur + sécurité
Corriger les oublis	Filtre <code>owner=request.user</code> manquant sur suppression
Ajuster la sécurité	Correction du <code>share_url</code> pointant vers l'API au lieu du frontend
Valider par les tests	Aucune intégration sans tests passants

Apport mesuré

- **x3 sur la vitesse d'implémentation** des vues et sérializers
- **Limite** : supervision indispensable sur la sécurité et les cas limites

SLIDE 10 — Conclusion

Ce qui a été livré

- ✓ Application web fonctionnelle et démontrable
- ✓ Architecture découplée front/back professionnelle
- ✓ Sécurité solide (JWT, hachage, isolation, UUID)
- ✓ 38 tests, 100% de couverture
- ✓ Documentation complète (README, OpenAPI, TESTING, SECURITY, PERF, MAINTENANCE)
- ✓ Containerisation Docker — installation en 1 commande

Prochaines étapes post-MVP

- Stockage S3 (scalabilité)
 - Rate limiting sur l'authentification
 - Tokens en httpOnly cookies (sécurité XSS)
 - Envoi de liens par email
 - Application mobile (API déjà prête)
-

Merci

Questions ?

<http://localhost:5173> — démo live